```
#include<stdio.h>
#include<stdlib.h>
struct node{
   int data;
   struct node* next;
};
void enqFront(struct node** front, struct node** rear, int value);
void enqRear(struct node** front, struct node** rear, int value);
void deqFront(struct node** front, struct node** rear);
void deqRear(struct node** front, struct node** rear);
void display(struct node* front);
int main()
{
   int n, value;
   struct node* front = NULL;
   struct node* rear = NULL;
   while(1)
   {
      printf("\n1. Enqueue at front");
      printf("\n2. Enqueue at rear");
      printf("\n3. Dequeue at front");
      printf("\n4. Dequeue at rear");
      printf("\n5. Display");
      printf("\n6. Exit\n");

      printf("Enter your choice: ");
      scanf("%d",&n);

      switch(n)
      {
```

```
        case 1:
          printf("Enter value: ");
          scanf("%d",&value);
          enqFront(&front,&rear,value);
          break;
        case 2:
          printf("Enter value: ");
          scanf("%d",&value);
          enqRear(&front,&rear,value);
          break;
        case 3:
          deqFront(&front,&rear);
          break;
        case 4:
          deqRear(&front,&rear);
          break;
        case 5:
          display(front);
          break;
        case 6:
          exit(0);
        default:
          printf("Invalid choice\n");
      }
  }
}
void enqFront(struct node** front, struct node** rear, int value)
{
  struct node* newnode = (struct node*)malloc(sizeof(struct node));
  newnode->data = value;
  newnode->next = *front;
```

```
    if(*front == NULL)
    {
        *front = *rear = newnode;
    }
    else
    {
        newnode->next = *front;
        *front = newnode;
    }
}

void enqRear(struct node** front, struct node** rear, int value)
{
    struct node* newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = value;
    newnode->next = NULL;
    if(*rear == NULL)
    {
        *front = *rear = newnode;
    }
    else
    {
        (*rear)->next = newnode;
        *rear = newnode;
    }
}

void deqFront(struct node** front, struct node** rear)
{
    if(*front == NULL)
    {
```

```
        printf("Deque is empty\n");
        return;
    }
    struct node* temp = *front;
    printf("Deleted element: %d\n", temp->data);
    *front = (*front)->next;
    if(*front == NULL)
        *rear = NULL;
    free(temp);
}
void deqRear(struct node** front, struct node** rear)
{
    if(*rear == NULL)
    {
        printf("Deque is empty\n");
        return;
    }
   if(*front == *rear)
    {
        printf("Deleted element: %d\n", (*rear)->data);
        free(*rear);
        *front = *rear = NULL;
        return;
    }
    struct node* temp = *front;
    while(temp->next != *rear)
        temp = temp->next;
    printf("Deleted element: %d\n", (*rear)->data);
    free(*rear);
    temp->next = NULL;
    *rear = temp;
```

```
}
void display(struct node* front)
{
   if(front == NULL)
   {
      printf("Queue is empty\n");
      return;
   }
   struct node* temp = front;
   while(temp != NULL)
   {
      printf("%d ",temp->data);
      temp = temp->next;
   }
   printf("\n");
}
```

```
1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 1
Enter value: 10

1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 1
Enter value: 20

1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 2
Enter value: 30

1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 5
20 10 30

1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 3
Deleted element: 20
```

```
1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 5
10 30

1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 4
Deleted element: 30

1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 4
Deleted element: 10

1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 5
Queue is empty

1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 6
```